



Functions

ER. PIYUSH PANT

MAHENDRA RATNA CAMPUS

C++

Inline Function



An **inline function** is a function where the compiler attempts to expand the function's body at the point of each function call instead of performing a traditional function call.

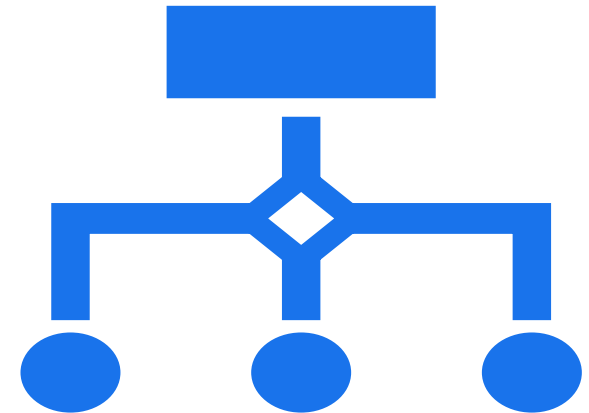


Purpose: To reduce overhead of function calls in performance-critical code.

Example

```
inline int multiply(int a, int b) {  
    return a * b;  
}
```

```
int main() {  
    int result = multiply(3, 4); // Compiler replaces with: int  
    result = 3 * 4;  
    cout << result;  
}
```





Inline Functions

When to use:

- Short functions (1–2 lines).
- Frequently used functions.

When not to use:

- Recursive functions.
- Functions with loops or switch statements.
- Very large functions (causes code bloat).

Function Overloading

Function overloading means having **multiple functions with the same name** but **different parameter lists** (number, types, or both).

Purpose

- Improves code readability.
- Helps perform similar operations for different data types.

Example

```
add(3, 4);      // Calls int version
add(3.5f, 2.5f); // Calls float version
add(1, 2, 3);   // Calls 3-parameter version
```

```
int add(int a, int b) {
    return a + b;
}
```

```
float add(float a, float b) {
    return a + b;
}
```

```
int add(int a, int b, int c) {
    return a + b + c;
}
```



Rules

- Same function name.
- Different number or type of parameters.
- Compiler selects the correct function based on arguments.

Invalid Overloads

```
int show();
```

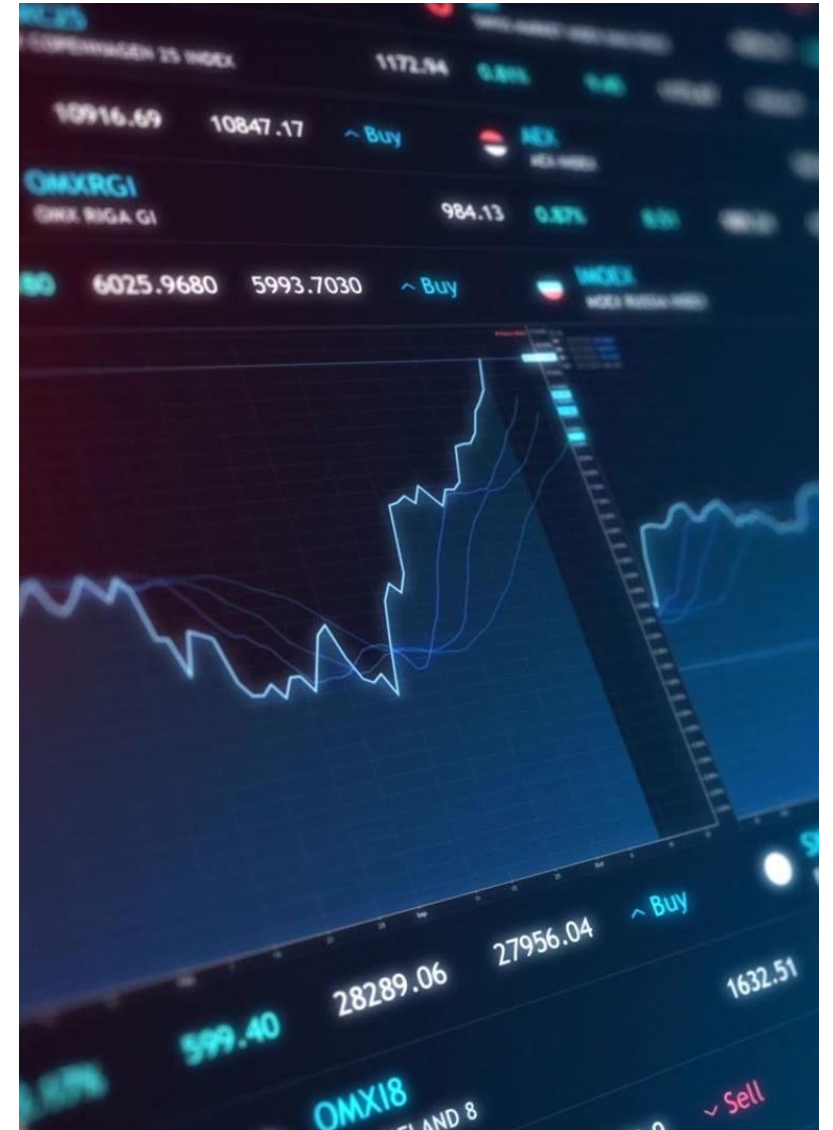
```
float show(); // ERROR: return type alone cannot  
distinguish overloads
```

Default Arguments

Default arguments allow **a function to be called with fewer arguments** than declared. The missing values are automatically filled with **default values**.

Syntax

```
void display(int a, int b = 10);
```



Example

```
#include <iostream>

using namespace std;

void display(int a, int b = 10, int c = 20) {
    cout << "a: " << a << ", b: " << b << ", c: " << c << endl;
}

int main() {
    display(1);      // b = 10, c = 20
    display(1, 2);   // c = 20
    display(1, 2, 3); // All provided
    return 0;
}
```

Rules for Default Argument

Default values must be assigned **from right to left**.

You cannot skip a default argument in the middle.

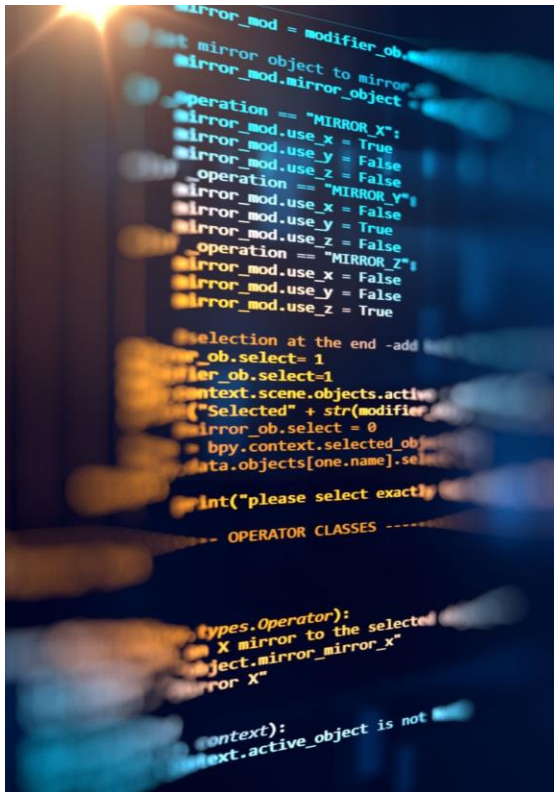
Correct

```
void example(int x, int y = 5, int z = 10);
```

Incorrect

```
void example(int x = 5, int y, int z = 10); // y does not have default
```

Pass by Reference (Reference Arguments)



Passing arguments by **reference** means the function works **on the original variables**, not on their copies.

```
void update(int &ref);
```

Reference arguments allow a function to modify the original variables by passing them by reference (using &).



Code

```
void swap(int &a, int &b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

```
int main() {  
    int x = 5, y = 10;  
    swap(x, y);  
    cout << "x: " << x << ", y: " << y;  
}
```

- Declared using the & symbol in the function parameter.
- No copy is made; the function works with the original variable.
- Useful for modifying values or improving performance.

When to Use:

When you want the function to **change the actual value**.

When you want to **avoid copying large data** (like arrays or objects).

Advantage



Saves memory and time.



Allows functions to **modify original data.**



Useful for large data structures (e.g., arrays, objects).

Pass By Value vs reference

Basis	Call by Value	Call by Reference
What is passed	A copy of the actual value	The address (reference) of the actual value
Changes in function	Do not affect the original variable	Affect the original variable
Memory usage	More (copies are created)	Less (no copies, direct reference)
Safety	Safer, original data is protected	Riskier, original data can be modified



Return by Reference

- Returning by reference means a function returns the address (reference) of a variable instead of just its value. This allows the caller to directly access and modify the original variable through the returned reference.

When to use

- You need to modify an existing variable directly.
- Avoiding unnecessary copies of large objects for efficiency.
- Implementing `operator[]` or `operator=` in custom classes.

Example

```
#include <iostream>

using namespace std;

int x = 5; // Global variable

int& give() {
    return x; // Returning reference to global
variable , return address
}

int main() {
    give() = 100; // Changes x directly
    cout << x;    // Output: 100
    return 0;
}
```



Recursive Function

- A recursive function in C++ is a function that calls itself to solve smaller instances of a problem. Recursion is widely used in problems like factorial computation, Fibonacci series, and tree traversal.

```
#include <iostream>
using namespace std;
```

```
int factorial(int n) {
    if (n <= 1) // Base case
        return 1;
    else
        return n * factorial(n - 1); // Recursive call
}
```

```
int main() {
    int num = 5;
    cout << "Factorial of " << num << " is: " << factorial(num) << endl;
    return 0;
}
```